

4. Git-HO

Git Hands-on Lab 4 Report – Merge Conflict Resolution Using P4Merge

To create a merge conflict between the **master** branch and a new branch, resolve the conflict using **P4Merge**, update the **.gitignore** file, and complete the merge successfully.

Prerequisites:

- Git installed and configured.
- Existing **GitLab** repository.
- P4Merge installed and configured as the default merge tool.
- Existing Git repository with previous commits.

Step 1: Verify Repository Status

Open Git Bash and verify that the repository is in a clean working state.

git status

```
HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Step 2: Create and Switch to a New Branch

Create a new branch named **GitWork** and switch to it.

git branch GitWork

git checkout GitWork

```
HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git branch GitWork

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git branch
  GitWork
* master
```

Step 3: Create hello.xml

Create a file named **hello.xml** with sample content.

echo "<message>Hello from GitWork Branch</message>" > hello.xml

Verify the repository status.

git status

```
HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git checkout GitWork
Switched to branch 'GitWork'

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (GitWork)
$ git branch
* GitWork
  master
```

Step 4: Stage and Commit the File

Add the file to the staging area and commit the changes.

```
git add hello.xml
```

```
git commit -m "Added hello.xml in GitWork"
```

Verify the repository status.

```
git status
```

```
HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (GitWork)
$ echo "<message>Hello from GitWork Branch</message>" > hello.xml

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (GitWork)
$ cat hello.xml
<message>Hello from GitWork Branch</message>

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (GitWork)
$ git status
On branch GitWork
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello.xml

nothing added to commit but untracked files present (use "git add" to track)
```

Step 5: Switch Back to Master

Switch back to the **master** branch.

```
git checkout master
```

Create another **hello.xml** file with different content.

```
echo "<message>Hello from Master Branch</message>" > hello.xml
```

Stage and commit the file.

```
git add hello.xml
```

```
git commit -m "Added hello.xml in master"
```

```

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (Gitwork)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (Gitwork)
$ git commit -m "Added hello.xml in Gitwork"
[Gitwork ad5e96f] Added hello.xml in Gitwork
1 file changed, 1 insertion(+)
create mode 100644 hello.xml

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (Gitwork)
$ git status
On branch Gitwork
nothing to commit, working tree clean

```

Step 6: View Commit History

Display the commit history.

`git log --oneline --graph --decorate --all`

```

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (Gitwork)
$ git checkout master
Switched to branch 'master'

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ echo "<message>Hello from Master Branch</message>" > hello.xml

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ cat hello.xml
<message>Hello from Master Branch</message>

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git commit -m "Added hello.xml in master"
[master 6fe30bf] Added hello.xml in master
1 file changed, 1 insertion(+)
create mode 100644 hello.xml

```

Step 7: Compare the Branches

Compare the differences between the **master** branch and **GitWork**.

`git diff master GitWork`

```

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git log --oneline --graph --decorate --all
* 6fe30bf (HEAD -> master) Added hello.xml in master
| * ad5e96f (GitWork) Added hello.xml in Gitwork
|/
* caaa663 Added branch.txt in GitNewBranch
* a3748af Added .gitignore
* 15f4d2e Added hello.txt

```

Step 8: View Differences Using P4Merge

Open **P4Merge** to visualize the differences between the two branches.

`git difftool master GitWork`

```

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git diff master GitWork
diff --git a/hello.xml b/hello.xml
index e3bbec4..234c289 100644
--- a/hello.xml
+++ b/hello.xml
@@ -1,1 @@
- <message>Hello from Master Branch</message>
+ <message>Hello from GitWork Branch</message>

```

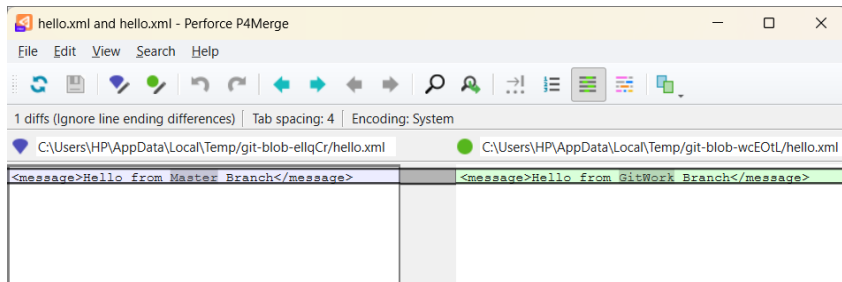
Step 9: Create a Merge Conflict

Merge the **GitWork** branch into the **master** branch.

```
git merge GitWork
```

A merge conflict occurs because both branches contain different versions of **hello.xml**.

```
git status
```

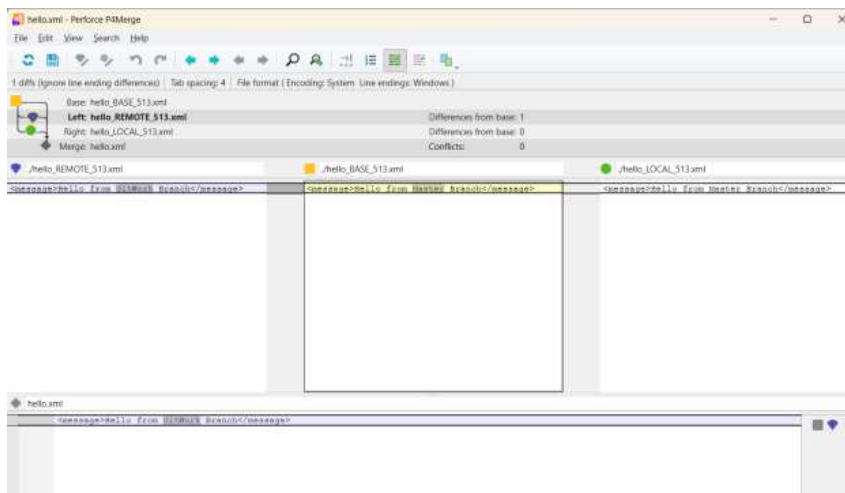


Step 10: Resolve the Merge Conflict

Resolve the merge conflict using the **P4Merge 3-way Merge Tool**.

```
git mergetool
```

After resolving the conflict, save the merged file and close **P4Merge**.



Step 11: Update .gitignore

Open the .gitignore file in Notepad++ and add the entry *.orig to ignore backup files created during merge conflict resolution



Step 12: Commit the Merge

Complete the merge by committing the resolved changes.

```
git commit -m "Resolved merge conflict and updated .gitignore"
```

Verify the repository status.

```
git status
```

```
HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master|MERGING)
$ git add hello.xml .gitignore

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master|MERGING)
$ git commit -m "Resolved merge conflict and updated .gitignore"
[master add6d4e] Resolved merge conflict and updated .gitignore

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Step 13: Delete the Merged Branch

Display the available branches and delete the merged branch.

```
git branch
```

```
git branch -d GitWork
```

```
HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git branch
  GitWork
* master

HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master)
$ git branch -d GitWork
Deleted branch GitWork (was ad5e96f).
```

Step 14: View Final Commit History

Display the final commit history.

git log --oneline --graph --decorate

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git log --oneline --graph --decorate
*   add6d4e (HEAD -> master) Resolved merge conflict and updated .gitignore
| \
|  * ad5e96f Added hello.xml in GitWork
| * | 6fe30bf Added hello.xml in master
| /
*   caaa663 Added branch.txt in GitNewBranch
*   a3748af Added .gitignore
*   15f4d2e Added hello.txt
```

Result

The merge conflict between the **master** branch and **GitWork** was created successfully and resolved using **P4Merge**. The **.gitignore** file was updated to ignore backup files, the merge was completed successfully, the **GitWork** branch was deleted after merging, and the repository was verified to be in a clean working state.

```
HP@DESKTOP-R2OG1F5 MINGW64 ~  
$ cd ~/GitLab  
  
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)  
$ pwd  
/c/Users/HP/GitLab  
  
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)  
$ git status  
On branch master  
nothing to commit, working tree clean  
  
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)  
$
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)  
$ git branch GitWork  
  
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)  
$ git branch  
GitWork  
* master
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)  
$ git checkout GitWork  
Switched to branch 'GitWork'  
  
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitWork)  
$ git branch  
* GitWork  
master
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitWork)
$ echo "<message>Hello from GitWork Branch</message>" > hello.xml

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitWork)
$ cat hello.xml
<message>Hello from GitWork Branch</message>
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitWork)
$ git status
On branch GitWork
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello.xml

nothing added to commit but untracked files present (use "git add" to track)
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitWork)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitWork)
$ git commit -m "Added hello.xml in GitWork"
[GitWork ad5e96f] Added hello.xml in GitWork
1 file changed, 1 insertion(+)
create mode 100644 hello.xml

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitWork)
$ git status
On branch GitWork
nothing to commit, working tree clean
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitWork)
$ git checkout master
Switched to branch 'master'

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ echo "<message>Hello from Master Branch</message>" > hello.xml

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ cat hello.xml
<message>Hello from Master Branch</message>
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git commit -m "Added hello.xml in master"
[master 6fe30bf] Added hello.xml in master
1 file changed, 1 insertion(+)
create mode 100644 hello.xml

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git log --oneline --graph --decorate --allv
fatal: unrecognized argument: --allv
```

```

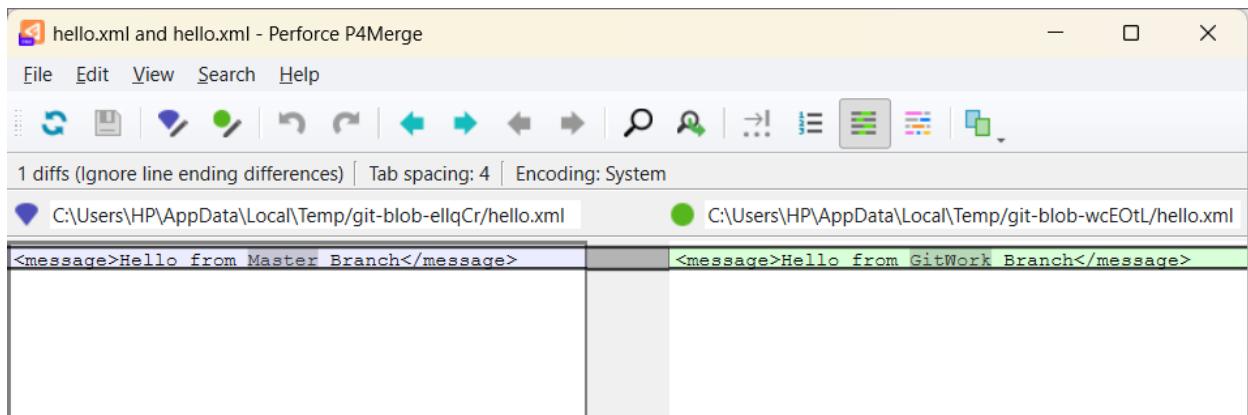
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git log --oneline --graph --decorate --all
* 6fe30bf (HEAD -> master) Added hello.xml in master
| * ad5e96f (GitWork) Added hello.xml in GitWork
|/
* caaa663 Added branch.txt in GitNewBranch
* a3748af Added .gitignore
* 15f4d2e Added hello.txt

```

```

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git diff master GitWork
diff --git a/hello.xml b/hello.xml
index e3bbec4..234c289 100644
--- a/hello.xml
+++ b/hello.xml
@@ -1,1 @@
- <message>Hello from Master Branch</message>
+ <message>Hello from GitWork Branch</message>

```



```

git merge GitWork
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
Y

git merge GitWork
Y

git merge GitWork
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git difftool master GitWorkY
fatal: ambiguous argument 'GitWorkY': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'

git merge GitWork

git merge GitWork

git merge GitWork
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$

git merge GitWork

git merge GitWork
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$

git merge GitWork
git merge GitWork
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$
git merge GitWorkgit merge GitWork
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git merge GitWorkgit merge GitWork
merge: GitWorkgit - not something we can merge

```

```

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git branch
  GitWork
* master

```

```

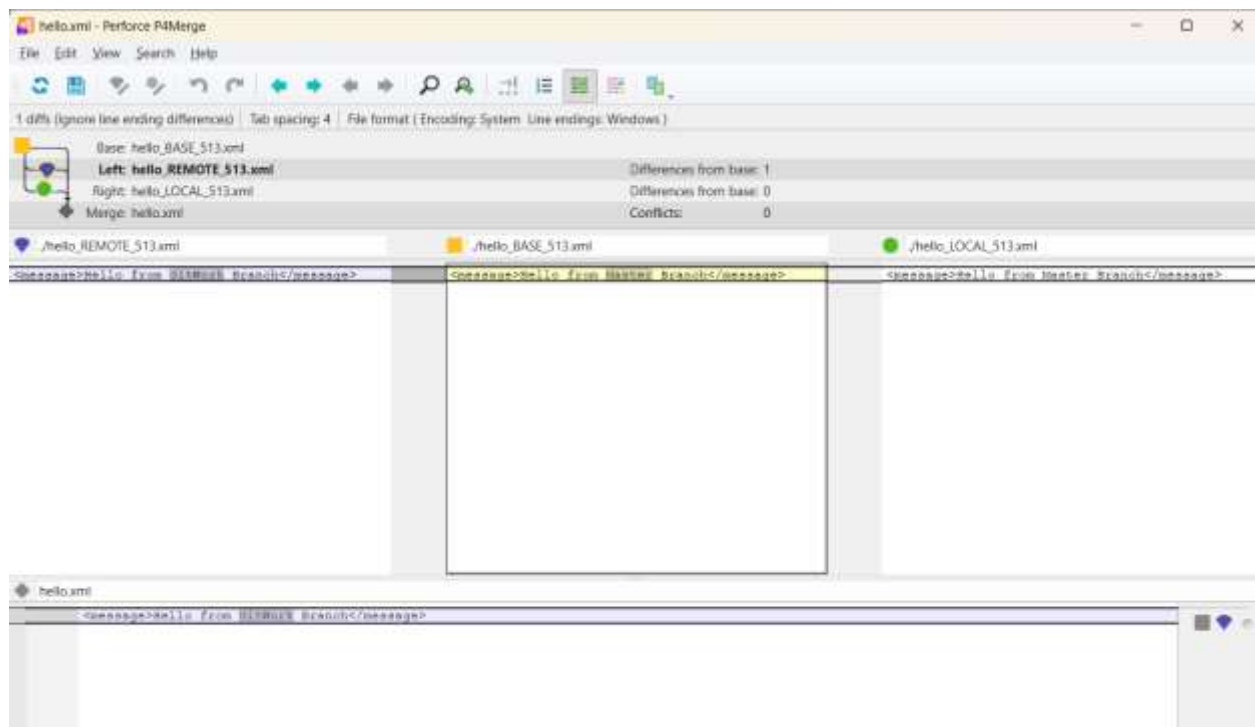
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git merge GitWork
Auto-merging hello.xml
CONFLICT (add/add): Merge conflict in hello.xml
Automatic merge failed; fix conflicts and then commit the result.

```

```
HP@DESKTOP-R20G1F5 MINGW64 ~/GitLab (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both added:      hello.xml

no changes added to commit (use "git add" and/or "git commit -a")
```



```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both added:   hello.xml

no changes added to commit (use "git add" and/or "git commit -a")

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master|MERGING)
$ git mergetool
Merging:
hello.xml

Normal merge conflict for 'hello.xml':
  {local}: created file
  {remote}: created file
error: invalid path './hello_BASE_513.xml'

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master|MERGING)
$ git status
On branch master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   hello.xml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello.xml.orig
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master|MERGING)
$ git status
On branch master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   hello.xml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  hello.xml.orig

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master|MERGING)
$ "C:/Program Files/Notepad++/notepad++.exe" .gitignore

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master|MERGING)
$ git add hello.xml .gitignore

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master|MERGING)
$ git commit -m "Resolved merge conflict and updated .gitignore"
[master add6d4e] Resolved merge conflict and updated .gitignore
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git status
On branch master
nothing to commit, working tree clean
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git branch
  GitWork
* master
```

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git branch -d GitWork
Deleted branch GitWork (was ad5e96f).
```